

Trabalho Vivencial de Estruturas de Dados: Implementação de um Autenticador Digital Utilizando Estruturas de Dados

Lysandre Alef Rodrigues Campos
Matrícula: 2524887
Centro de Ciências Tecnológicas
(CCT)
Universidade de Fortaleza
(UNIFOR) Fortaleza, Brasil

Lucas Oliveira Silva
Matrícula: 2524988
Centro de Ciências Tecnológicas
(CCT)
Universidade de Fortaleza
(UNIFOR) Fortaleza, Brasil

Erika dos Anjos Silva
Matrícula: 2524952
Centro de Ciências Tecnológicas
(CCT)
Universidade de Fortaleza
(UNIFOR) Fortaleza, Brasil

Luiz Gonzaga Sa Neto
Matrícula: 2526575
Centro de Ciências Tecnológicas
(CCT)
Universidade de Fortaleza
(UNIFOR) Fortaleza, Brasil

Francisco Eder Santos Gualberto
Matrícula: 2513734
Centro de Ciências Tecnológicas
(CCT)
Universidade de Fortaleza
(UNIFOR) Fortaleza, Brasil

Francisco das Chagas Silva
Barbosa
Matrícula: 2526818
Centro de Ciências Tecnológicas
(CCT)
Universidade de Fortaleza
(UNIFOR) Fortaleza, Brasil

INTRODUÇÃO:

A estrutura de dados do tipo **Pilha** (stack) é muito importante em vários programas de computador, principalmente quando a gente trabalha com coisas em ordem e precisa controlar como o código roda. No contexto deste projeto de autenticador digital, a pilha tem um papel essencial ao guardar temporariamente as árvores AVL que são criadas a partir de cada linha do texto. A pilha ajuda a organizar o processamento das estruturas de forma ordenada, seguindo o princípio LIFO (Last In, First Out), garantindo que as árvores precisam ser processadas na ordem inversa da sua inserção. as árvores assim processadas garantem que os códigos HASH do documento sejam gerados corretamente, como o projeto determina. A pilha funciona como um meio que garante a integridade da sequência de operações e ajuda a tornar o processo de autenticação digital mais confiável.

CONCEITO:

A pilha é uma estrutura de dados linear que organiza os elementos de forma sequencial. Seu funcionamento é baseado no princípio LIFO (Last In, First Out), ou seja, o último elemento inserido será o primeiro a ser removido. Para facilitar o entendimento, podemos imaginar uma pilha como uma pilha de pratos: você sempre coloca um prato no topo e também retira pelo topo. As principais operações realizadas em uma pilha são:

- Empilhar (push): adicionar um elemento no topo
- Desempilhar (pop): remover o elemento do topo
- Consultar o topo (peek): visualizar o elemento do topo sem removê-lo

APLICAÇÃO DA PILHA NO PROJETO:

No desenvolvimento do autenticador digital, a pilha é utilizada após a criação das árvores AVL. O processo funciona da seguinte forma: cada linha do arquivo de texto é lida e transformada em uma árvore AVL. Após isso, essa árvore é inserida na pilha. Esse procedimento é repetido para todas as linhas do texto. Ao final, teremos uma pilha contendo várias árvores, cada uma representando uma linha do documento. **Etapas do processo:** Após a criação da árvore AVL para cada linha do texto: A árvore é inserida na pilha. Fluxo será:

1. Lê uma linha do arquivo
2. Gera lista de palavras
3. Cria árvore AVL
4. Empilha a árvore
5. Repete para todas as linhas

UTILIZAÇÃO DA PILHA NESSE PROJETO:

A escolha da pilha nesse projeto não é por acaso. Ela é ideal porque: permite processar os dados na ordem inversa, organiza as árvores de forma simples e eficiente, facilita o controle do fluxo do algoritmo e atende exatamente ao que foi solicitado no enunciado do trabalho.

Processamento das Árvores (Desempilhamento):

Depois que todas as árvores são inseridas na pilha, inicia-se a etapa de processamento. Nesse momento, as árvores são retiradas da pilha uma por uma. Para cada árvore removida, é gerado um código hash utilizando a lógica definida no projeto. Esse processo continua até que a pilha esteja completamente vazia. Ao final, cada linha do texto terá gerado um hash, e todos esses valores são exibidos separadamente.

Lógica do Funcionamento (Pseudocódigo):

A lógica geral da pilha dentro do projeto pode ser representada da seguinte forma:

Agora a explicação detalhada de cada bloco:

Pilha<ArvoreAVL> pilha = new Pilha<>() Declara a pilha usando Generics. O **<ArvoreAVL>** garante que só árvores AVL podem ser empilhadas — sem misturar tipos. Internamente usa uma lista encadeada de nós.

para cada linha → dividirPalavras(linha) O `BufferedReader` lê o arquivo uma linha por vez. Cada linha passa pelo `split(" ")` que quebra o texto pelos espaços, e cada palavra é inserida no fim da `ListaDinamica`.

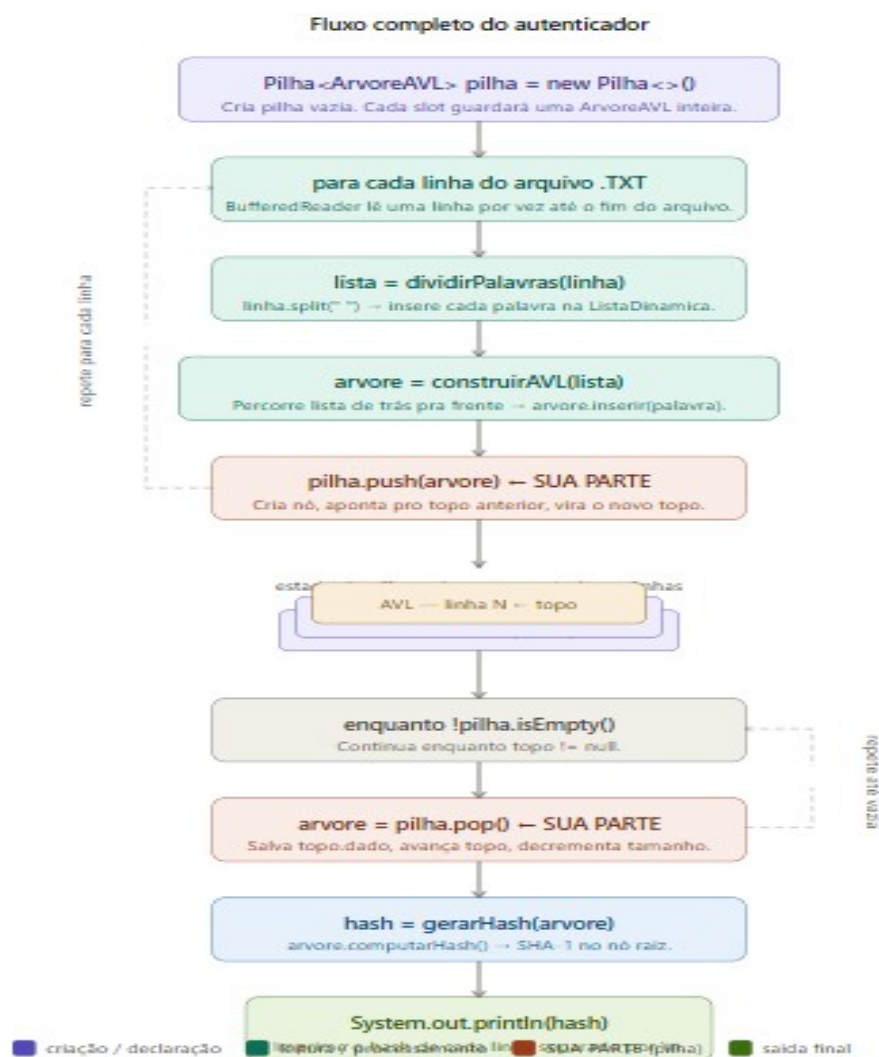
construirAVL(lista) Percorre a lista de trás pra frente (do índice `getTamanho() - 1` até `0`) chamando `arvore.inserir(palavra)`. A AVL cuida do balanceamento e descarta duplicatas via `compareToIgnoreCase() == 0`.

`pilha.push(arvore)` ← **sua parte** Cria um novo nó encadeado, faz ele apontar para o antigo topo, e esse nó vira o novo topo. A última linha lida fica sempre no topo da pilha.

`while (!pilha.isEmpty())` Verifica se `topo == null`. Enquanto houver nós, continua desempilhando. O loop processa as árvores na ordem inversa à que foram empilhadas (LIFO).

`pilha.pop()` ← **sua parte** Salva `topo.dado` (a árvore), avança o ponteiro `topo` para o próximo nó, decrementa o contador e retorna a árvore salva.

gerarHash(arvore) → imprimir(hash) Chama `arvore.computarHash()` que percorre os nós calculando SHA-1 de forma recursiva (filhos → pai). O hash do nó raiz é o resultado final, impresso com `println` separando cada linha por `\n`.

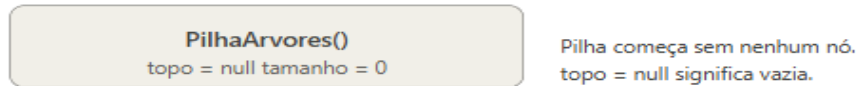


INTEGRAÇÃO COM O PROJETO:

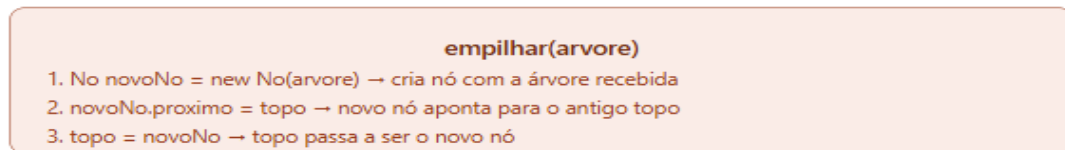
Classe interna No — cada nó da pilha



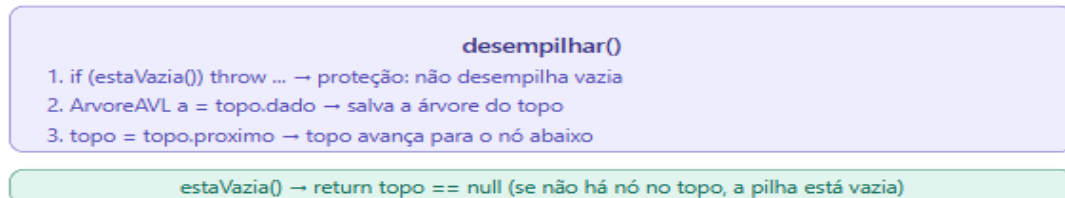
Construtor — cria pilha vazia



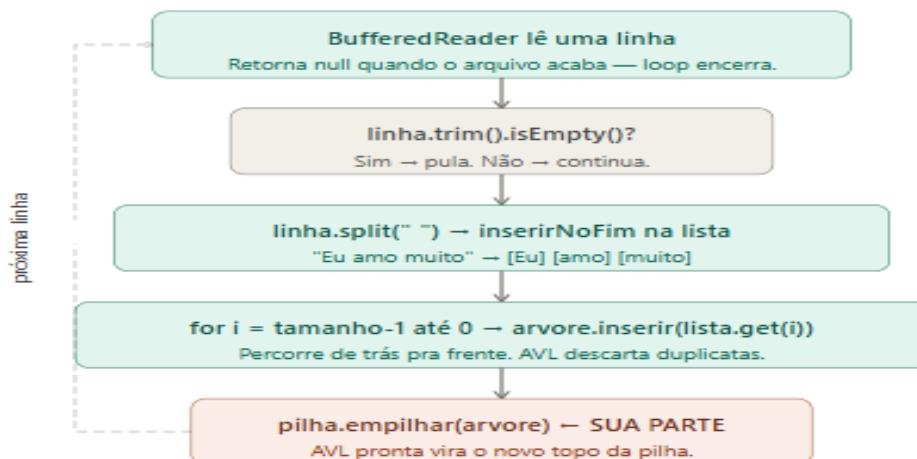
empilhar(arvore) — push, sua parte



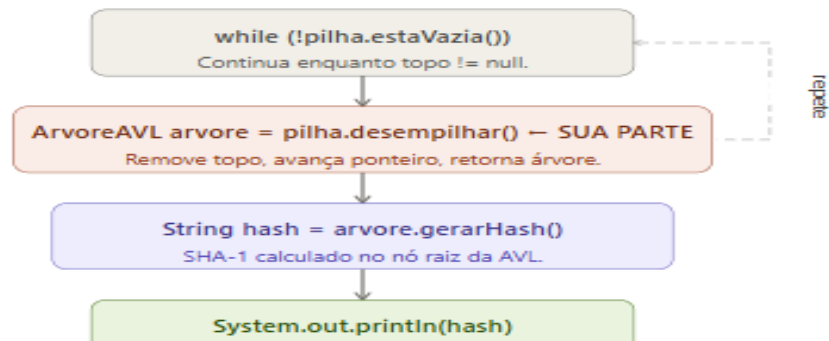
desempilhar() — pop, sua parte



Fase 1 — leitura e construção da pilha



Fase 2 — processamento da pilha



PilhaArvores.java:

No (classe interna) — cada nó guarda uma `ArvoreAVL` no campo `dado` e um ponteiro `proximo` que aponta para o nó abaixo. É a corrente que forma a pilha.

`PilhaArvores()` — inicializa `topo = null` e `tamanho = 0`. Pilha começa sem nenhum nó.

`Empilhar(arvore)` — cria um novo nó, faz ele apontar para o antigo topo, e esse nó vira o novo topo. A última árvore inserida fica sempre no topo.

`Esempilhar()` — lança exceção se vazia, salva `topo.dado`, avança `topo` para o próximo nó e retorna a árvore.

`EstaVazia()` — verifica `topo == null`. Simples e direto.

Autenticador.java:

Fase 1 — lê cada linha, divide em palavras com `split(" ")`, insere na lista dinâmica, percorre ao contrário para montar a AVL, e chama `pilha.empilhar(arvore)`.

Fase 2 — entra no `while (!pilha.estaVazia())`, chama `pilha.desempilhar()` para pegar a árvore do topo, chama `arvore.gerarHash()` para obter o SHA-1, e imprime com `println`.